

ltglide: il glissando del tone-burst di Linkwitz

Diario di studio — porting C++ e plugin per la suite SEAM-LTM

Giuseppe Silvi — SEAM

2026

Sommario

Questo documento è il diario di studio di *ltglide*, il generatore di glissando del tone-burst sagomato di Linkwitz. Racconta il porting in C++ della specifica Faust `seam.linkwitz.lib` (prefisso `slw`), già discussa in dettaglio nel diario di *ltburst*, e la costruzione del plugin che la ospita. La teoria del burst sagomato e la derivazione della specifica Faust restano nel diario di *ltburst* (`plugins/ltburst/doc/study/ltburst-study.tex`), che questo documento richiama per riferimento.

Indice

1	Il glissando come generalizzazione del burst fisso	1
2	Il motore a grana retriggerata (Approccio A)	2
3	I due modi di temporizzazione e la guardia di sicurezza	3
4	Il transport della passata	3
5	Rinvii	4

1 Il glissando come generalizzazione del burst fisso

Il tone-burst sagomato di Linkwitz [1] fissa il numero di cicli $N = 5$ e lascia libera la frequenza portante f_0 , ottenendo un segnale a Q costante che *ltburst* riproduce fedelmente a frequenza singola. *ltglide* generalizza questo schema facendo scorrere la frequenza portante lungo una passata, così lo strumento diventa un rilievo continuo esteso su tutta la banda coperta dallo sweep. Uno sweep esterno, calcolato dalla funzione `sweepfreq` della libreria `seam.linkwitz.lib`, produce il progresso di frequenza in funzione del tempo di passata. Un motore a grana consuma quello sweep con un campionamento-e-tenuta a ogni proprio inizio, così ciascuna grana resta a frequenza singola per tutta la propria durata. Il risultato è una sequenza di burst di Linkwitz via via più acuti o più gravi, ciascuno internamente identico a un burst di *ltburst* alla propria frequenza tenuta. Questa costruzione rende *ltglide* uno strumento di misura continuo: la stessa banda-costante di un singolo burst si ripete lungo tutto lo spettro coperto dalla passata.

La funzione `sweepfreq` offre due leggi di percorrenza, già specificate in Faust e riportate fedelmente nel core C++ (`ltglide_dsp.h`):

$$\text{sweepfreq}(f_0, f_1, \text{smode}, p) = \begin{cases} f_0 + (f_1 - f_0)p, & \text{smode} = 0 \text{ (lineare)} \\ f_0 \left(\frac{f_1}{f_0}\right)^p, & \text{smode} = 1 \text{ (esponenziale)} \end{cases} \quad (1)$$

dove $p \in [0, 1]$ è il progresso della passata. La legge lineare percorre hertz uguali per unità di progresso, e il suo punto medio $p = 0.5$ vale la media aritmetica $(f_0 + f_1)/2$. La legge

esponenziale percorre ottave uguali per unità di progresso, e il suo punto medio vale la media geometrica $\sqrt{f_0 f_1}$. La suite sceglie l'esponenziale come modalità predefinita, coerente con la percezione logaritmica dell'altezza e con la scala delle frequenze già usata dai controlli F0/F1 del plugin.

2 Il motore a grana retriggerata (Approccio A)

Il cuore di *ltglide* è *GlissBurst*, il porting diretto della funzione Faust *glissburst* già derivata e discussa nel diario di *lthburst* (§5.3, Approccio A). Il motore porta avanti due stati accoppiati campione per campione: una rampa di grana *phase_* in $[0, 1)$ e una frequenza tenuta *fg_*. Ogni campione avanza la rampa di un incremento pari al reciproco del periodo di grana corrente, calcolato dalla frequenza tenuta del campione precedente. Quando la rampa supera l'unità, oppure al primissimo campione della passata, il motore dichiara un nuovo inizio di grana. Proprio a ogni inizio di grana la frequenza tenuta *fg_* si aggiorna al valore corrente dello sweep esterno *fsig*, e resta fissa a quel valore per tutta la durata della grana successiva: questo è il campionamento-e-tenuta descritto nella Sezione 1. La portante e la finestra di Hann derivano entrambe dalla stessa coppia (*phase_*, *fg_*) appena aggiornata, così condividono un unico riferimento di fase e restano sincrone per costruzione.

```
glissburst(N,delta,dmode,fsig) = sin(2*ma.PI*u) * win
with {
  phase = grain : _,!;
  fg     = grain : !,_;
  grain = loop ~ (_,_)
  with {
    loop(pphase,pfg) = nphase, nfg
    with {
      den      = max(20.0, pfg);
      Tg       = select2(dmode, max(delta, N/den), N/den + delta);
      inc      = 1.0/max(1.0, Tg*ma.SR);
      adv      = pphase + inc;
      onset    = (adv >= 1.0) | (1 - 1');
      nphase   = adv - floor(adv);
      nfg      = select2(onset, pfg, fsig);
    };
  };
  den = max(20.0, fg);
  Tg  = select2(dmode, max(delta, N/den), N/den + delta);
  u   = fg * phase * Tg;
  win = (u < N) * (0.5 - 0.5*cos(2*ma.PI*u/N));
};
```

Il porting C++ riproduce questa struttura ricorsiva in modo identico: `GlissBurst::process(fsig)` calcola prima la coppia successiva (*phase_*, *fg_*) a partire dai valori precedenti, esattamente come il blocco *loop* Faust, e solo dopo usa quella coppia aggiornata per sintetizzare portante e finestra nello stadio di uscita. Una conseguenza diretta di questa struttura riguarda il primissimo campione della passata: la rampa avanza di un incremento *prima* di essere usata nello stadio di uscita dello stesso campione, così il primo campione della grana resta a una piccola distanza misurabile dallo zero-crossing. Il test *doctest* verifica infatti che il primo campione resti entro 3.8×10^{-6} dallo zero, con un confronto a tolleranza assoluta, perché una tolleranza relativa attorno a un valore atteso esattamente nullo degenera a una tolleranza pressoché nulla. Questa piccola distanza dallo zero è un'eredità fedele della definizione ricorsiva Faust, e differisce dalla convenzione a contatore esatto (*c* come stato) usata da *lthburst*, già discussa e motivata nel suo diario di studio.

Una conseguenza numerica di questa costruzione lega *ltglide* a *ltburst* in modo verificabile. A frequenza costante il motore a grana retriggerata degenera a un generatore a periodo fisso, e il modo *gap* con $f = 1000$ Hz, $\delta = 0.3$ s produce inizi di grana distanziati di $N/f + \delta = 0.305$ s, cioè 14640 campioni a 48 kHz. Questo è esattamente il periodo del burst fisso di *ltburst* a $f_0 = 1000$ Hz con *dwell* = 0.3 s, già validato nel suo diario e nel suo record di validazione. Il numero coincide per costruzione, perché entrambi i motori condividono la stessa formula di periodo quando la frequenza è costante, e questa coincidenza numerica costituisce un'ancora di verifica tra i due oggetti pur restando due implementazioni C++ indipendenti, ciascuna con codice proprio nel rispettivo plugin.

3 I due modi di temporizzazione e la guardia di sicurezza

Il motore offre due leggi per il periodo di grana T_g , già specificate in Faust e riprodotte identiche nel porting C++ tramite il parametro **Timing** del plugin. Il modo *passo* fissa la distanza inizio-inizio a $T_g = \max(\delta, N/f_g)$, così il passo tra le grane resta costante e il silenzio residuo si accorcia quando la frequenza tenuta sale. Il modo *gap* fissa il silenzio fine-inizio a $T_g = N/f_g + \delta$, così la distanza inizio-inizio cresce quando la frequenza tenuta scende, perché la grana stessa dura più a lungo. Entrambe le leggi condividono la stessa guardia di sicurezza $\text{den} = \max(20 \text{ Hz}, f_g)$, che protegge il calcolo di T_g quando la frequenza tenuta è ancora nulla, al primissimo inizio della passata, oppure quando lo sweep raggiunge il proprio estremo grave. Questa guardia rende il modo *passo* sicuro contro la sovrapposizione delle grane anche al limite inferiore dello sweep, perché a 20 Hz una grana di cinque cicli dura già 250 ms, un valore che il termine $\max(\delta, N/f_g)$ rispetta sempre per costruzione.

4 Il transport della passata

Il plugin colloca il motore a grana dentro una passata delimitata da due impulsi Dirac, gestita dalla classe **GlideTransport** e indipendente dal motore stesso. Una passata inizia con un impulso di testa, seguito da un tratto di silenzio (*lead*) di cinque secondi, che lascia assestare l'ambiente di misura prima che lo sweep abbia inizio. Segue la fase di glissando vera e propria, della durata impostata dal parametro **Sweep Time**, durante la quale il progresso p cresce da zero a uno e alimenta **sweepfreq** e quindi il motore a grana. Al termine dello sweep un secondo tratto di silenzio (*tail*) di cinque secondi precede l'impulso di coda, simmetrico all'impulso di testa e utile a delimitare la registrazione della passata. Il parametro **Loop**, se attivo, fa seguire all'impulso di coda un'attesa di due secondi e poi un nuovo impulso di testa, così la passata si ripete in modo automatico e deterministico. Il metodo di avvio **trigger()** del transport ignora le richieste che arrivano mentre una passata è già in corso, così il transport mantiene una passata alla volta e la sua struttura resta prevedibile per uno strumento di misura. Il plugin espone **Loop** come unico comando di transport, e a esso affida l'avvio e la ripetizione delle passate. Un pulsante di avvio istantaneo, provato in fase di collaudo, consegna in alcuni host la propria pressione come un singolo valore, quello finale a zero, e per questo il comando **Loop** a valore stabile governa il transport in modo affidabile su ogni host. Il metodo **trigger()** resta nel motore SDK-free, dove il collaudo automatico lo verifica e la fase di ricevitore potrà riusarlo.

Il loop serve principalmente alla fase di ricevitore, ancora da progettare (fase 3b-ii della suite), dove più passate consecutive verranno mediate per migliorare il rapporto segnale-rumore della misura. Gli impulsi Dirac di testa e di coda offrono a quel ricevitore un riferimento temporale netto per allineare e segmentare le passate prima della media, un ruolo analogo a quello che gli stessi impulsi svolgono già come marcatori di inizio e fine registrazione per un operatore umano.

5 Rinvii

La misura degli spettri della passata, il confronto con le figure originali di Linkwitz e l'analisi delle risposte in frequenza ricavate dal glissando restano compiti della fase di ricevitore. Questo diario si ferma alla validazione del motore a grana e del transport come generatore standalone, già documentata nel record di validazione (`plugins/ltglide/doc/ltglide-validation.md`). La teoria del tone-burst sagomato, la derivazione della specifica Faust e la discussione della differenza tra le convenzioni di stato di *ltburst* e di *ltglide* restano nel diario di *ltburst*, che questo documento richiama come fonte primaria.

Riferimenti bibliografici

[1] Siegfried Linkwitz. Shaped tone-burst testing. *J. Audio Eng. Soc.*, 28(4):250–258, April 1980.